🧠 Ways to refer to a commit

(Formas de referenciar um commit)

👉 Em termos simples:

Você precisa dizer ao Git **“qual versão do projeto você quer olhar”**

E existem várias formas de fazer isso.

🔑 Ideia central

Um **commit** pode ser identificado de várias maneiras:

- por nome (branch)
- por posição (HEAD~1)
- por código (hash)
- por referência remota

🧬 1. HEAD — onde você está agora

📌 Exemplo:

```
git diff HEAD
```

🧠 O que é?

👉 HEAD = o commit atual

💡 Tradução

“o ponto onde você está agora na linha do tempo”

🔧 Exemplo

```
git show HEAD
```

👉 mostra o último commit

⏪ 2. HEAD~1, HEAD~2 — voltando no tempo

📌 Exemplo:

```
git diff HEAD~1 HEAD
```

🧠 O que significa?

Comando	Significado
HEAD	agora
HEAD~1	1 commit atrás
HEAD~2	2 commits atrás

🔧 Exemplo visual

```
C1 — C2 — C3 — C4 (HEAD)
```

- HEAD = C4
- HEAD~1 = C3
- HEAD~2 = C2

🔀 3. Nome da branch

📌 Exemplo:

```
git diff main
```

🧠 O que é?

👉 Uma branch é um “ponteiro” para um commit

🔧 Exemplo

```
git diff main login
```

👉 compara duas linhas de desenvolvimento

💡 Uso real

- comparar feature com main
- ver o que mudou antes de merge

🏷️ 4. Tag (versões do projeto)

📌 Exemplo:

```
git diff v1.0
```

🧠 O que é?

👉 Tags são versões marcadas

Ex:

- v1.0
- v2.0

💡 Uso real

- comparar versões do sistema
- deploy
- releases

🔢 5. Hash do commit (ID único)

📌 Exemplo:

```
git show 3e887ab
```

🧠 O que é?

👉 Cada commit tem um código único

Ex:

```
3e887ab4f91c2a8c...
```

🔧 Uso

Você pode usar só o começo:

```
git show 3e887ab
```

💡 Uso real

- investigar bug
- revisar commit específico

🌍 6. Branch remota

📌 Exemplo:

```
git diff origin/main
```

🧠 O que é?

👉 versão do projeto no servidor (GitHub)

🔧 Uso

```
git diff main origin/main
```

👉 compara:

- seu código local
- código online

🧠 Visual completo

```
Referências possíveis:

HEAD            → atual
HEAD~1          → anterior
main            → branch local
origin/main     → branch remota
v1.0            → versão (tag)
3e887ab         → commit específico
```

🔥 Exemplo completo (vida real)

```
git log --oneline
```

Saída:

```
a1b2c3 versão 3
d4e5f6 versão 2
g7h8i9 versão 1
```

Comparações:

```
git diff HEAD~1 HEAD
git show a1b2c3
git diff d4e5f6 a1b2c3
```

🧠 Cenário real (nível profissional)

Você precisa descobrir um bug:

👉 “quando isso quebrou?”

Estratégia:

```
git diff versão_antiga versão_nova
```

Ou:

```
git show commit_específico
```

⚠️ Erros comuns

❌ confundir HEAD com branch

→ gera erro de interpretação

❌ não usar HEAD~ corretamente

→ pega commit errado

🎯 Regra de ouro

Sempre saiba **em qual ponto da história você está trabalhando**

🚀 Nível engenharia

Agora você consegue:

- navegar no tempo do projeto
- comparar versões
- investigar mudanças
- entender evolução do código